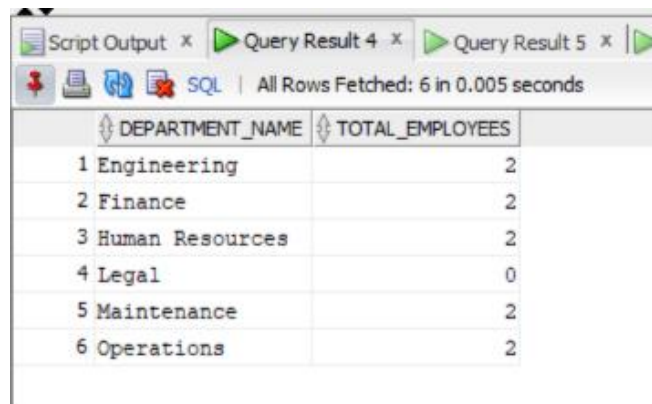


Section 04 — Aggregation Functions

Total Employees per Department

```
SELECT jd.name AS department_name,  
       COUNT(e.emp_ID) AS total_employees  
FROM JOB_DEPARTMENT jd  
LEFT JOIN EMPLOYEE e ON jd.job_ID = e.job_ID  
GROUP BY jd.name  
ORDER BY jd.name;
```

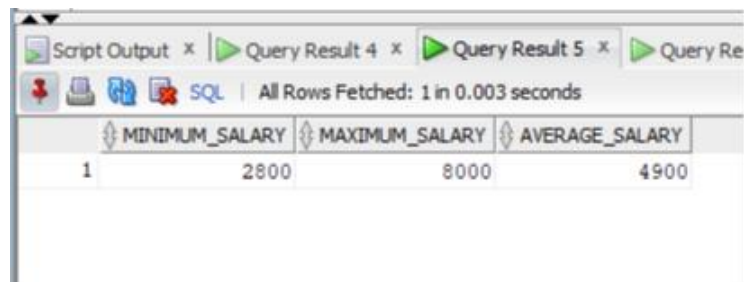


The screenshot shows a SQL query result window with a table containing 6 rows. The columns are DEPARTMENT_NAME and TOTAL_EMPLOYEES. The data is as follows:

	DEPARTMENT_NAME	TOTAL_EMPLOYEES
1	Engineering	2
2	Finance	2
3	Human Resources	2
4	Legal	0
5	Maintenance	2
6	Operations	2

Minimum, Maximum & Average Salary

```
SELECT MIN(amount) AS minimum_salary,  
       MAX(amount) AS maximum_salary,  
       AVG(amount) AS average_salary  
FROM SALARY_BONUS;
```

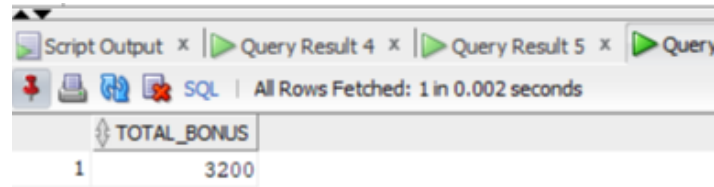


The screenshot shows a SQL query result window with a table containing 1 row. The columns are MINIMUM_SALARY, MAXIMUM_SALARY, and AVERAGE_SALARY. The data is as follows:

	MINIMUM_SALARY	MAXIMUM_SALARY	AVERAGE_SALARY
1	2800	8000	4900

Total Bonus Paid

```
SELECT SUM(bonus) AS total_bonus  
FROM SALARY_BONUS;
```



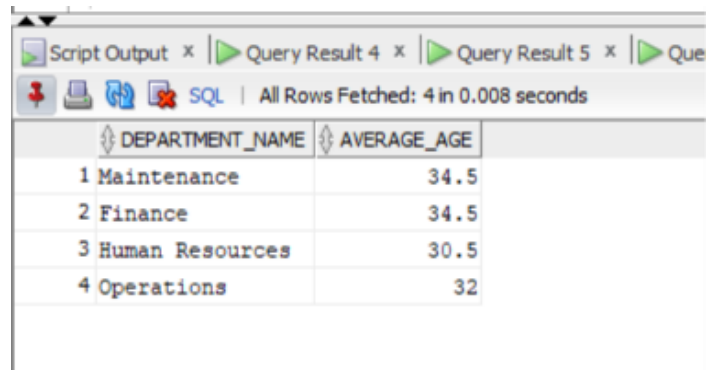
The screenshot shows a SQL query result window with a toolbar at the top containing icons for script output, query results, and a play button. The text 'All Rows Fetched: 1 in 0.002 seconds' is displayed. Below the toolbar, a table with one column 'TOTAL_BONUS' is shown, containing a single row with the value '3200'.

TOTAL_BONUS
3200

Section 04 — GROUP BY with HAVING

Departments Where Average Age > 30

```
SELECT jd.name AS department_name,  
       AVG(e.age) AS average_age  
FROM JOB_DEPARTMENT jd  
JOIN EMPLOYEE e ON jd.job_ID = e.job_ID  
GROUP BY jd.name  
HAVING AVG(e.age) > 30;
```

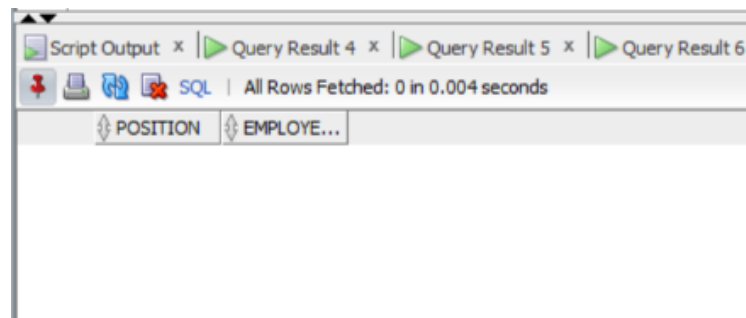


The screenshot shows a SQL query result window with the following data:

	DEPARTMENT_NAME	AVERAGE_AGE
1	Maintenance	34.5
2	Finance	34.5
3	Human Resources	30.5
4	Operations	32

Positions Shared by More Than 2 Employees

```
SELECT position,  
       COUNT(emp_ID) AS employee_count  
FROM QUALIFICATION  
GROUP BY position  
HAVING COUNT(emp_ID) > 2;
```



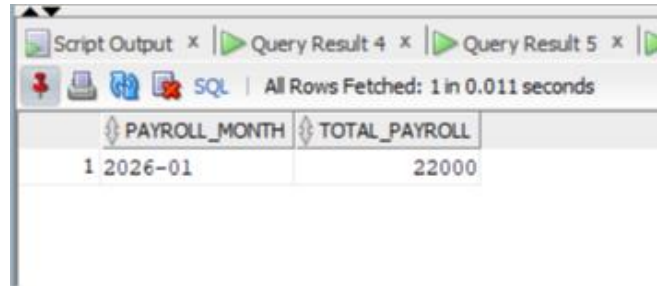
The screenshot shows a SQL query result window with the following headers and no data rows:

POSITION	EMPLOYEE...
----------	-------------

No position is held by more than 2 employees, so the result is empty.

Months Where Total Payroll > 20,000

```
SELECT TO_CHAR(pay_date, 'YYYY-MM') AS payroll_month,  
       SUM(total_amount) AS total_payroll  
FROM PAYROLL  
GROUP BY TO_CHAR(pay_date, 'YYYY-MM')  
HAVING SUM(total_amount) > 20000;
```



The screenshot shows a SQL query result window with a single row of data. The window has a title bar with tabs for 'Script Output', 'Query Result 4', and 'Query Result 5'. Below the tabs, there are icons for a red pin, a printer, a refresh, and a red 'X' over a document, followed by the text 'All Rows Fetched: 1 in 0.011 seconds'. The main area of the window displays a table with two columns: 'PAYROLL_MONTH' and 'TOTAL_PAYROLL'. The first row of data shows '1 2026-01' for the payroll month and '22000' for the total payroll.

PAYROLL_MONTH	TOTAL_PAYROLL
1 2026-01	22000